

UNIVERZITET U TUZLI
FAKULTET ELEKTROTEHNIKE



SEMINARSKI RAD

Prepoznavanje uzoraka

Klasifikacija Indijskog voća

Profesor: Dr.sc. Damir Demirović vanr.prof.

Student:

Tuzla, maj 2023

Sažetak

U ovom seminarskom radu ćemo govoriti o klasifikaciji indijskih voćaka tačnije o načinu na koji se to radi. Voće su plodovi kulturih ili samoniklih voćaka, koji se upotrebljavaju u prehrani u svježem ili suhom stanju i u obliku prerađevina (kompoti, džemovi, marmelade, slatko, sokovi i dr.). Klasifikacija je rađena na šest vrsta indijskog voća jabuka, banana, guava, limeta, narandza i nar. Klasifikacija voća urađena je po moću konvolucijske neuronske mreže.

Ključne riječi: konvolucijska neuronska mreža, duboko učenje, klasifikacija voća, detekcija voća, mašinsko učenje...

Abstract

In this seminar paper, we will talk about the classification of Indian fruit trees, more specifically about the way it is carried out. Fruits are the fruits of cultivated or wild fruit trees that are used in food in fresh or dry form and in the form of processed products (compotes, jams, marmalades, sweets, juices, etc.). The classification was made on six types of Indian fruits: apples, bananas, guavas, limes, oranges and pomegranates. Fruit classification was done using a convolutional neural network.

Keywords: convolutional neural network, deep learning, fruit classification, fruit detection, machine learning...

Sadržaj:

Uvod.....	6
Data set	7
Duboko učenje	8
Konvolucijske neuronske mreže	9
Rekurentna neuronska mreža.....	10
Duboka mreža uvjerenja	11
Skup podataka Indian Fruits Dataset with Quality	11
TensorFlow biblioteka.....	12
Struktura neuronske mreže korištene u eksperimentu	13
Rješavanje problema.....	13
Implementacija neuronske mreže	16
Treniranje neuronske mreže	17
Plotiranje performanse neuronske mreže	19

Popis slika:

Slika 1. Tropsko voće	6
Slika 2. Struktura neuronske mreže	8
Slika 3. Tehnike učenja.....	9
Slika 4. Osnovna arhitektura CNN-a	10
Slika 5. Specifikacije telefona.....	12
Slika 6. Struktura konvolucijske neuronske mreže.....	13
Slika 7. Voće za testiranje.....	15
Slika 8. Pregled modela konvolucijske neuronske mreže.....	17
Slika 9. Preciznost i gubici neuronske mreže.....	20
Slika 10. Matrica konfuzije	21

Uvod

Voće je zbirna imenica koja podrazumijeva plodove ili sjemenke najčešće višegodišnjih stabala i grmova koji se uglavnom mogu jesti prijesni. Botanička razlika između voća i povrća je ta da je voće sjemenkasta struktura koja se razvija iz oplođenog cvijeta, dok povrće nastaje iz drugih dijelova biljke, poput korijena i lišća. Paprike, rajčica, tikvice, patlidžan, grašak, bamija, maslina, bundeva i krastavci su isto plodovi koji pripadaju voću, ali pošto nisu slatki niti kiseli, kulinarски se mstraju i pripremaju kao povrće. Dinja i lubenica smatraju se voćem ali botančki su povrće.

Voće su plodovi kultiviranih voćaka ili samonikliх višegodišnjih biljaka namijenjenih ljudskoj prehrani u svježem ili prerađenom stanju. Uspijeva na svim kontinentima i u svim klimatskim područjima gdje za pojedine vrste postoji odgovarajući sastav tla i klimatski uvjeti. Od svih namirnica biljnog porijekla voće se, zbog svoje boje i oblika, mirisa, dobrog okusa, biološke i prehrambene vrijednosti nalazi na jelovnicima od najstarijih vremena do danas, a najviše se cijeni zbog svog hemijskog sastava.

Maseni udio vode, u kojoj je otoplјena većina voćnih sastojaka, iznosi 70 do 90%. Voda u voću ima važnu fiziološku ulogu jer aktivira lučenje u želucu i crijevima, te povoljno djeluje na bubrege i kožu. Svi sastojci voća, osim vode, čine njegovu suhu tvar, a mogu biti toplјivi i netoplјivi u voćnom soku.

Energetska vrijednost voća nije velika jer manji udio otpada na energetske sastojke. Najhranjiviji i najvredniji sastojci voća su šećeri. Ljudski ih organizam najlakše apsorbira pa voće ne samo što osvježava, već je i lako probavljivo. U voću ima i celuloze. Iako je neprobavljiva, ona vrlo dobro djeluje na rad crijeva.

Voće se razlikuje prema vremenu sazrijevanja, stupnju sazrijevanja, namjeni i prema porijeklu proizvodnje.



Slika 1. Tropsko voće

Data set

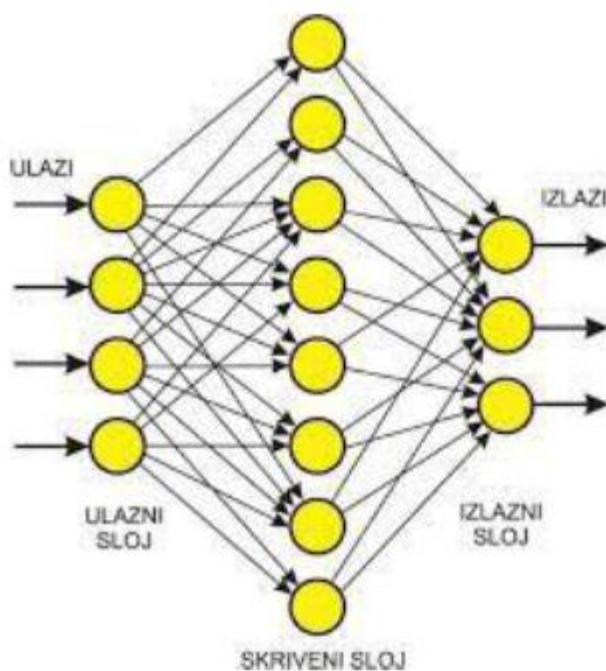
U ovom seminarskom radu koristit ćemo data set sa [linka](#), te ćemo navesti neke osnovne karakteristike istog. Skup podataka sadrži 14700 slika od 6 vrsta indijskog voća i povrća te se stalno ažurira. Postotni udio dobiti na tržištu voća značajan je u odnosu na ukupnu poljoprivrednu proizvodnju. U agroindustriji je najpotrebnija brza i tačna klasifikacija voća. Voće se može klasificirati u različite klase prema njihovim vanjskim značajkama kao što su oblik, veličina i boja korištenjem neke računalne vizije i tehnika dubokog učenja [1], [2], [3], [4]. Skup podataka FruitNet stvoren je kako bi uključio indijsko voće zajedno s njegovim parametrima kvalitete za ono koje se intenzivno konzumira ili izvozi prema . Sastoji se od šest klasa indijskog voća, a to su jabuka, banana, guava, limeta, naranča i šipak. Dalje su kategorizirani u kvalitetu, lošu kvalitetu i mješovitu kvalitetu. Slike voća snimljene su s različitom pozadinom, u različitim svjetlosnim uvjetima u zatvorenom i vanjskom okruženju. Slika 1 prikazuje uzorke slika u skupu podataka koji se sastoji od slika snimljenih u različitim okruženjima.

Visokokvalitetne slike voća potrebne su za rješavanje problema klasifikacije i prepoznavanja voća. Za izradu modela strojnog učenja, uredan i čist skup podataka osnovni je zahtjev. S tim smo ciljem stvorili skup podataka šest popularnih indijskih vrsta voća pod nazivom "FruitNet". Ovaj skup podataka sastoji se od 14700+ visokokvalitetnih slika 6 različitih klasa voća u obrađenom formatu. Slike su podijeljene u 3 podmape

- 1) Kvalitetno voće
- 2) Voće loše kvalitete
- 3) Miješano kvalitetno voće.

Svaka podmapa sadrži 6 slika voća, tj. jabuke, banane, guave, limete, naranče i nara. Za snimanje slika korišten je mobilni telefon s kamerom visoke rezolucije. Slike su snimljene na različitim pozadinama i u različitim uvjetima osvjetljenja.

Većina postojećih skupova podataka sa slikama sadrži i objekt i „noisy“ pozadinu. To bi moglo dovesti do slučajeva u kojima će promjena pozadine dovesti do netačne klasifikacije objekta. Postoji više prednosti ovog projekta, zadatka identificiranja voća. S jedne strane, voće ima određene kategorije koje je teško razlikovati, poput roda citrusa, koji sadrži narandže i grejp. Također smo trenirali duboku neuronsku mrežu koja je sposobna identificirati plodove iz slika. Ovo je dio složenijeg projekta koji ima za cilj dobivanje klasifikatora koji može identificirati mnogo širi niz objekata iz slika. To se uklapa u trenutni trend tvrtki koje rade u području proširene stvarnosti. Trenutno se identifikacija objekata temelji na dubokoj neuronskoj mreži. Takva bi mreža imala brojne primjene na višestrukim mrežama poput autonomne navigacije, modeliranja objekata, upravljanja procesima ili interakcije čovjeka i robota.



Slika 2. Struktura neuronske mreže

Duboko učenje

U području prepoznavanja i klasifikacije slika najuspješniji rezultati dobiveni su korištenjem umjetnih neuronskih mreža. Ove mreže čine osnovu za većinu modela dubokog učenja. Duboko učenje je klasa algoritama strojnog učenja koji koriste više slojeva koji sadrže nelinearne jedinice za obradu. Svaka razina uči transformirati svoje ulazne podatke u nešto apstraktniji i kompozitniji prikaz. Dokazano je da duboke neuronske mreže - posebno konvolucijske neuronske mreže - postižu sjajne rezultate u području prepoznavanja slika.



Slika 3. Tehnike učenja

Neuronska mreža je skup algoritama koji nastoje prepoznati temeljne odnose u skupu podataka kroz proces koji oponaša način rada ljudskog mozga. Zbog toga se umjetne neuronske mreže odnose na organske ili umjetne sustave neurona. Umjetne neuronske mreže mogu se prilagoditi promjenama ulaznih podataka, te zbog toga umjetna neuronska mreža može generirati najbolji mogući rezultat bez potrebe za redizajniranjem izlaznih kriterija. Umjetne neuronske mreže mogu se smatrati pojednostavljenim modelima mreža od neurona koji se javlja u mozgu od živih bića. Gledajući s biološkog stajališta glavna zadaća neuronske mreže je obrada i procesiranje podataka.

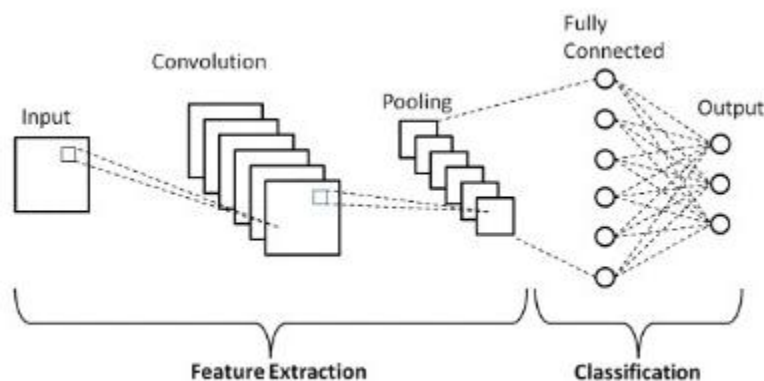
Konvolucijske neuronske mreže

Konvolucijske neuronske mreže (CNN) dio su modela dubokog učenja. Takva mreža može se sastojati od konvolucijskih slojeva, slojeva za udruživanje, ReLU slojeva, potpuno povezanih slojeva i slojeva gubitaka. U tipičnoj arhitekturi CNN-a, svaki konvolucijski sloj prati sloj Rectified Linear Unit (ReLU), zatim Pooling sloj, zatim jedan ili više konvolucijskih slojeva i na

kraju jedan ili više potpuno povezanih slojeva. Karakteristika koja razlikuje CNN od obične neuronske mreže je uzimanje u obzir strukture slika tijekom njihove obrade. Imajte na umu da obična neuronska mreža pretvara ulaz u jednodimenzionalni niz što uvježbani klasifikator čini manje osjetljivim na promjene položaja.

Konvolucija koristi tri važne ideje koje su motivisale istraživače a to su:

1. Rijetka interakcija
2. Dijeljenje parametara
3. Ekvivariantno predstavljanje.



Slika 4. Osnovna arhitektura CNN-a

Rekurentna neuronska mreža

Drugi algoritam dubokog učenja je rekurzivna neuronska mreža. U ovoj vrsti arhitekture isti skup pondera se rekurzivno primjenjuje na neke podatke. Tradicionalno su se rekurentne mreže koristile za obradu sekvencijalnih podataka, a najpoznatiji su primjeri prepoznavanje rukopisa ili govora. Korištenjem rekurentnih konvolucijskih slojeva s nekim maksimalnim slojevima bazena između njih i konačnim globalnim maksimalnim slojem bazena na kraju dobiva se nekoliko prednosti. Prvo, unutar sloja, svaka jedinica uzima u obzir stanje jedinica u sve većem području oko sebe. Drugo, korištenjem ponavljajućih slojeva, dubina mreže se povećava bez dodavanja više parametara. Rekurentne mreže pokazale su dobre rezultate u obradi prirodnog jezika.

Duboka mreža uvjerenja

Još jedan model koji je dio algoritama dubokog učenja je mreža dubokog učenja. Duboka mreža vjerovanja je vjerojatnostni model sastavljen od više slojeva skrivenih jedinica. Korištenje mreže dubokih uvjerenja je isto kao i druge predstavljene mreže, ali se također može koristiti za prethodnu obuku duboke neuronske mreže kako bi se poboljšale početne vrijednosti pondera. Ovaj proces je važan jer može poboljšati kvalitetu mreže i smanjiti vrijeme obuke. Mreže dubokih uvjerenja mogu se kombinirati s konvolucijskim kako bi se dobile konvolucijske mreže dubokih uvjerenja koje iskorištavaju prednosti koje nude obje vrste arhitektura.

Skup podataka Indian Fruits Dataset with Quality

Slike voća snimljene su Apple iPhoneom 6 sa stražnjom kamerom od 8 megapiksela, Z2 plus sa stražnjom kamerom od 13 megapiksela i Realme 5 pro sa stražnjom kamerom od 48 megapiksela. Sve slike skupa podataka izvorne veličine 3024×3024 promijenjene su u dimenzije 256×256 pomoću python skripte. Slike su u .jpg slikama. Slike dobivene u različitim uvjetima okoline kao što su različiti svjetlosni uvjeti, različita pozadina i iz različitih kutova. Nakon snimanja slike su organizirane u mape loše kvalitete, dobre kvalitete i mješovite kvalitete. Nadalje, svaka mapa kvalitete ima šest različitih mapa klasa voća, odnosno jabuke, banane, guave, limete, naranče i nara. Specifikacije uređaja koji se koriste za dobivanje slika i specifikacije dobivenih slika prikazane su u tablici.

Sr. No.	Camera Particulars	Details		
1	Camera maker	Apple	ZUK	Realme
2	Camera Model	iPhone 6	Z2 Plus	Realme 5 Pro
3	F-stop	f/2.2	f/2.2	f/1.8
4	Exposure time	1/25s	1/214s	1/33s
5	ISO Speed	ISO-250	ISO-100	ISO-1120
6	Exposure bias	0 step	0 step	0 step
7	Focal length	4mm	4mm	5mm
8	metering mode	Pattern	Centered Weighted Average	Unknown
9	Flash mode	No flash	No flash	No flash
10	35mm focal length	29	29	0

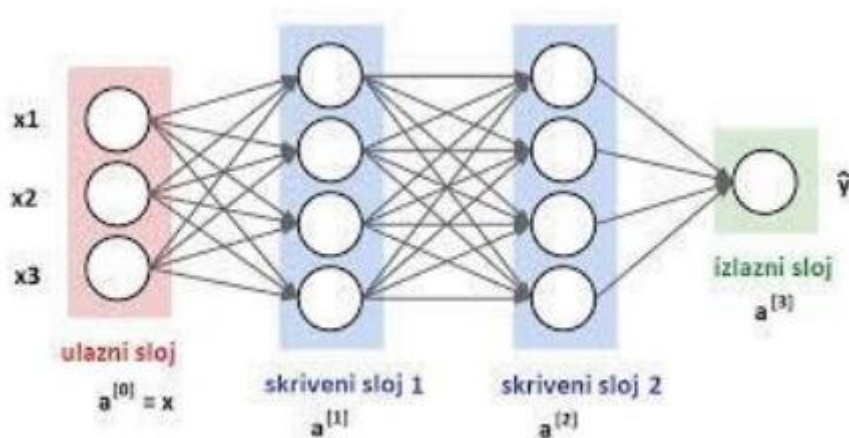
Slika 5. Specifikacije telefona

TensorFlow biblioteka

Za potrebe implementacije, obuke i testiranja mreže opisane u ovom radu koristili smo biblioteku TensorFlow. Tensorflow je open source platforma koja se koristi za strojno učenje. Tensorflow u sebi sadrži mnoštvo alata, biblioteka i resursa koji omogućuju nastanak snažnih aplikacija koje se zasnivaju na strojnom učenju. Google je jedna od kompanija koja koristi Tensorflow. Keras je API od TensorFlow-a 2.0 koje je lako dostupan te se bavi rješavanjem problema strojnog učenja modernim postupcima dubokog učenja. Glavne komponente u sustavu TensorFlow su klijent, koji koristi sučelje sesije za komunikaciju i jedan ili više procesa, pri čemu je svaki proces odgovoran za arbitražu pristupa jednom ili više računalnih uređaja (kao što su CPU jezgre ili GPU) i za izvršavanje čvorova grafa na tim uređajima prema uputama. TensorFlow nudi neke moćne značajke kao što su: omogućuje mapiranje računanja na više strojeva, za razliku od većine drugih ima ugrađenu podršku za automatsko izračunavanje gradijenta; može djelomično izvršavati podgrafove cijelog grafa i može dodati ograničenja uređajima, kao što je postavljanje čvorova na uređaje određenog tipa, osigurati da se dva ili više objekata smjeste u isti prostor itd.

Struktura neuronske mreže korištene u eksperimentu

Za ovaj projekt koristili smo konvolucijsku neuronsku mrežu. Kao i prethodno opisana ova vrsta mreže koristi konvolucione slojeve, objedinjene slojeve, ReLU slojeve, potpuno povezane slojeve i slojeve gubitaka. U tipičnom CNN-u arhitekture, svaki konvolucijski sloj sadrži ispravljenu linearnu jedinicu (ReLU), zatim pooling sloj, zatim jedan ili više konvolucijskih slojeva i jedan ili više potpuno povezanih slojeva. Karakteristika koja razlikuje CNN od obične neuronske mreže uzima u obzir strukturu slika dok ih obrađuje. Standardna neuronska mreža pretvara ulaz u jednodimenzionalni niz koji klasifikator čini manje osjetljivim na promjene položaja.



Slika 6. Struktura konvolucijske neuronske mreže

Rješavanje problema

Glavni cilj jeste koristiti moći konvolucijske neuronske mreže i Pythona izvršiti klasifikaciju voća.

Da bi poceli rjesavati problem u Python-u prvo moramo import-ovati potrebne biblioteke.

```
import numpy as np
import pandas as pd
import os
import time
import matplotlib.pyplot as plt
import cv2
import seaborn as sns
sns.set_style('darkgrid')
import shutil
from sklearn.metrics import confusion_matrix, classification_report
```

```

from sklearn.model_selection import train_test_split
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.layers import Dense, Activation, Dropout, Conv2D,
MaxPooling2D, BatchNormalization
from tensorflow.keras.optimizers import Adam, Adamax
from tensorflow.keras.metrics import categorical_crossentropy
from tensorflow.keras import regularizers
from tensorflow.keras.models import Model
# prevent annoying tensorflow warning
import logging
logging.getLogger("tensorflow").setLevel(logging.ERROR)
import warnings
warnings.simplefilter("ignore")

```

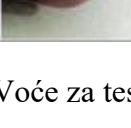
Sada trebamo da istražimo dataset koji sadrži slike dobre kvalitete, loše kvalitete i mješovite kvalitete.

Pomoću Python-a prikazujemo sve tri varijante :

```

bad_path=r'../input/fruitnet-indian-fruits-dataset-with-quality/Processed
Images_Fruits/Bad Quality_Fruits'
good_path=r'../input/fruitnet-indian-fruits-dataset-with-quality/Processed
Images_Fruits/Good Quality_Fruits'
mixed_path=r'../input/fruitnet-indian-fruits-dataset-with-quality/Processed
Images_Fruits/Mixed Quality_Fruits'

```

Name of the fruit	Good quality images		Bad quality images		Mixed quality images	
Apple						
Banana						
Guava						
Lime						
Orange						
Pomegranate						

Slika 7. Voće za testiranje

Sljedeće što trebamo uraditi jeste transformirati slike na fiksnu rezoluciju (226x226) te horizontalno okrenuti da možemo imati više slika za treniranje.

U našoj bazi imamo 3 seta slika :

1. Set za treniranje neuronske mreže
2. Set za validaciju – Ovo su slike koje će nam pomoći u zaključivanju da li naša neuronska mreža underfitta ili overfitta, sve dok se vrši treniranje i validacija u real-time.
3. Set za testiranje – Ovo su slike koje ćemo koristiti da provjerimo efikasnost naše neuronske mreže.

U sljedećem primjeru, pokušaćemo izbjegnuti overfitting mijenjajući naše slike. Pod „mijenjajući podrazumijevamo blage varijacije na slike tako da imamo više podataka, bez gubitka semantičkog značenja u našim podacima (parametri kao `horizontal_flip`, `channel_shift_range`, `rotation_range`, `zoom_range`).

Ova izmjene se vrše u metodi `ImageDataGenerator`.

```
trgen = ImageDataGenerator(
    horizontal_flip=True,
    rotation_range=20,
    width_shift_range=0.2,
    height_shift_range=0.2,
    zoom_range=0.2,
)
```

Implementacija neuronske mreže

Kao što smo naveli koristimo konvolucijsku neuronsku mrežu. Koristiti ćemo već napravljeni neuronski model te ćemo koristiti već pomenuti transfer learning, tj. istreniraćemo neuronsku mrežu za naše potrebe.

#Konvolucijska neuronska mreža

```
model = Sequential()
model.add(Dense(10, activation='relu', input_shape=img_shape))

model.add(Conv2D(input_shape=(224,224,3),filters=64,kernel_size=(3,3),padding="same", activation="relu"))
model.add(Conv2D(filters=64,kernel_size=(3,3),padding="same", activation="relu"))
model.add(Conv2D(filters=128, kernel_size=(3,3),padding="same", activation="relu"))
model.add(Conv2D(filters=128, kernel_size=(3,3),padding="same", activation="relu"))
model.add(MaxPool2D(pool_size=(2,2),strides=(2,2)))
model.add(Conv2D(filters=256, kernel_size=(3,3),padding="same", activation="relu"))
model.add(Conv2D(filters=256, kernel_size=(3,3),padding="same", activation="relu"))
model.add(Conv2D(filters=256, kernel_size=(3,3),padding="same", activation="relu"))
model.add(MaxPool2D(pool_size=(2,2),strides=(2,2)))
model.add(Conv2D(filters=512, kernel_size=(3,3),padding="same", activation="relu"))
model.add(Conv2D(filters=512, kernel_size=(3,3),padding="same", activation="relu"))
model.add(Conv2D(filters=512, kernel_size=(3,3),padding="same", activation="relu"))
model.add(MaxPool2D(pool_size=(2,2),strides=(2,2)))
model.add(Conv2D(filters=512, kernel_size=(3,3),padding="same", activation="relu"))
model.add(Conv2D(filters=512, kernel_size=(3,3),padding="same", activation="relu"))
model.add(Conv2D(filters=512, kernel_size=(3,3),padding="same", activation="relu"))
model.add(MaxPool2D(pool_size=(2,2),strides=(2,2)))
model.add(MaxPool2D(pool_size=(2,2),strides=(2,2)))
```



```
model.add(Flatten())
model.add(Dense(units=18, activation="sigmoid"))
```

Možemo vidjeti pregled modela :

Model: "sequential_4"

Layer (type)	Output Shape	Param #
dense_6 (Dense)	(None, 224, 224, 10)	40
conv2d_8 (Conv2D)	(None, 224, 224, 64)	5824
conv2d_9 (Conv2D)	(None, 224, 224, 64)	36928
conv2d_10 (Conv2D)	(None, 224, 224, 128)	73856
conv2d_11 (Conv2D)	(None, 224, 224, 128)	147584
max_pooling2d_3 (MaxPooling2D)	(None, 112, 112, 128)	0
conv2d_12 (Conv2D)	(None, 112, 112, 256)	295168
conv2d_13 (Conv2D)	(None, 112, 112, 256)	590080
conv2d_14 (Conv2D)	(None, 112, 112, 256)	590080
max_pooling2d_4 (MaxPooling2D)	(None, 56, 56, 256)	0
conv2d_15 (Conv2D)	(None, 56, 56, 512)	1180160
conv2d_16 (Conv2D)	(None, 56, 56, 512)	2359808
conv2d_17 (Conv2D)	(None, 56, 56, 512)	2359808
max_pooling2d_5 (MaxPooling2D)	(None, 28, 28, 512)	0
conv2d_18 (Conv2D)	(None, 28, 28, 512)	2359808
conv2d_19 (Conv2D)	(None, 28, 28, 512)	2359808
conv2d_20 (Conv2D)	(None, 28, 28, 512)	2359808
max_pooling2d_6 (MaxPooling2D)	(None, 14, 14, 512)	0
max_pooling2d_7 (MaxPooling2D)	(None, 7, 7, 512)	0
flatten_2 (Flatten)	(None, 25088)	0
dense_7 (Dense)	(None, 18)	451602
Total params: 15,170,362		
Trainable params: 15,170,362		
Non-trainable params: 0		

Slika 8. Pregled modela konvolucijske neuronske mreže

Treniranje neuronske mreže

Treniranje neuronske mreže pomoću EfficientNetB5

EfficientNetB5 je još jedan popularan model dubokog učenja koji se koristi za različite zadatke, uključujući klasifikaciju slika. Ovaj model se temelji na konceptu efikasnosti, gdje se traži optimalni omjer između performansi i resursa. Nakon što smo kreirali model, zamrzavamo težine baznog modela kako bismo zadržali prethodno naučene značajke. Zatim dodajemo dodatne

slojeve na vrh baznog modela kako bismo prilagodili model našem zadatku klasifikacije. Na kraju kompiliramo model s odgovarajućim gubitkom i metrikama te treniramo model na odgovarajućim skupovima podataka. Važno je napomenuti da je potrebno pripremiti skupove podataka za treniranje (train_data) i validaciju (val_data) u odgovarajućem obliku prije nego što se mogu koristiti za treniranje modela.

Mi u ovom slučaju radimo sa osnovnim modelom i stopom učenja od 0,01 i 10 epoch-a što znači da našem modelu dajemo 10 šansi da nauči naše uzorke. U primjeru, naš parametar patience (strpljenje) kaže modelu da prestane sa treniranjem ako nema napretka nakon 4 epoch-a, a monitor kaže modelu koju metriku da posmatra kako bi primjenio rano zaustavljanje (early stopping). Ukoliko je stopa učenja prevelika možemo je smanjiti pomoću funkcije rlronp npr. ako je početna stopa učenja 0.001, onda će se smanjiti na 0.0005 zbog toga što je factor=0.5.

```
rlronp=tf.keras.callbacks.ReduceLROnPlateau(monitor="val_loss", factor=0.5,
patience=2,verbose=1)
estop=tf.keras.callbacks.EarlyStopping(monitor="val_loss", patience=4,
verbose=1,restore_best_weights=True)
callbacks=[rlronp, estop]
history = model.fit(
    x=train_gen,
    epochs=epochs,
    verbose=1,
    validation_data=valid_gen,
    validation_steps=None,
    shuffle=False,
    initial_epoch=0,
    callbacks=callbacks
)
```

Rezultati

```
Epoch 1/10
164/164 - 112s 577ms/step - loss: 23.9557 - accuracy: 0.6017 - val_loss: 18.8450
- val_accuracy: 0.8895
Epoch 2/10
164/164 - 91s 552ms/step - loss: 14.4675 - accuracy: 0.8461 - val_loss: 11.3427 -
val_accuracy: 0.9447
Epoch 3/10
164/164 - 90s 551ms/step - loss: 9.3591 - accuracy: 0.9157 - val_loss: 7.3683 -
val_accuracy: 0.9611
Epoch 4/10
164/164 - 91s 553ms/step - loss: 6.2397 - accuracy: 0.9558 - val_loss: 4.9432 -
val_accuracy: 0.9672
Epoch 5/10
164/164 - 91s 554ms/step - loss: 4.2538 - accuracy: 0.9660 - val_loss: 3.3664 -
val_accuracy: 0.9744
Epoch 6/10
164/164 - 91s 553ms/step - loss: 2.9858 - accuracy: 0.9727 - val_loss: 2.3729 -
val_accuracy: 0.9836
Epoch 7/10
```

```

164/164 - 91s 553ms/step - loss: 2.1468 - accuracy: 0.9831 - val_loss: 1.7668 -
val_accuracy: 0.9846
Epoch 8/10
164/164 - 91s 553ms/step - loss: 1.6140 - accuracy: 0.9850 - val_loss: 1.3305 -
val_accuracy: 0.9867
Epoch 9/10
164/164 - 92s 563ms/step - loss: 1.2973 - accuracy: 0.9831 - val_loss: 1.0896 -
val_accuracy: 0.9816
Epoch 10/10
164/164 - 92s 558ms/step - loss: 1.0719 - accuracy: 0.9883 - val_loss: 0.8882 -
val_accuracy: 0.9846

```

Plotiranje performanse neuronske mreže

Ako nam je preciznost validacije/test veća nego preciznosti treniranja, to znači da smo uspjeli istrenirati model spram naših uzoraka/podataka. Međutim, ako imamo obrnuti slučaj, znači da je prisutan slučaj overfittinga. Što se tiče loss charta, idealna situacija jeste da imamo male gubitke, ali train (trenažni gubici) ne bi trebali biti manji od testnih/validacionih gubitaka.

Model Accuracy (Preciznost)

```

plt.plot(Epochs,tloss, 'r', label='Training loss')
plt.plot(Epochs,vloss,'g',label='Validation loss' )
plt.scatter(index_loss+1 +start_epoch,val_lowest, s=150, c= 'blue',
label=sc_label)
plt.set_title('Training and Validation Loss')
plt.set_xlabel('Epochs')
plt.set_ylabel('Loss')
plt.legend()

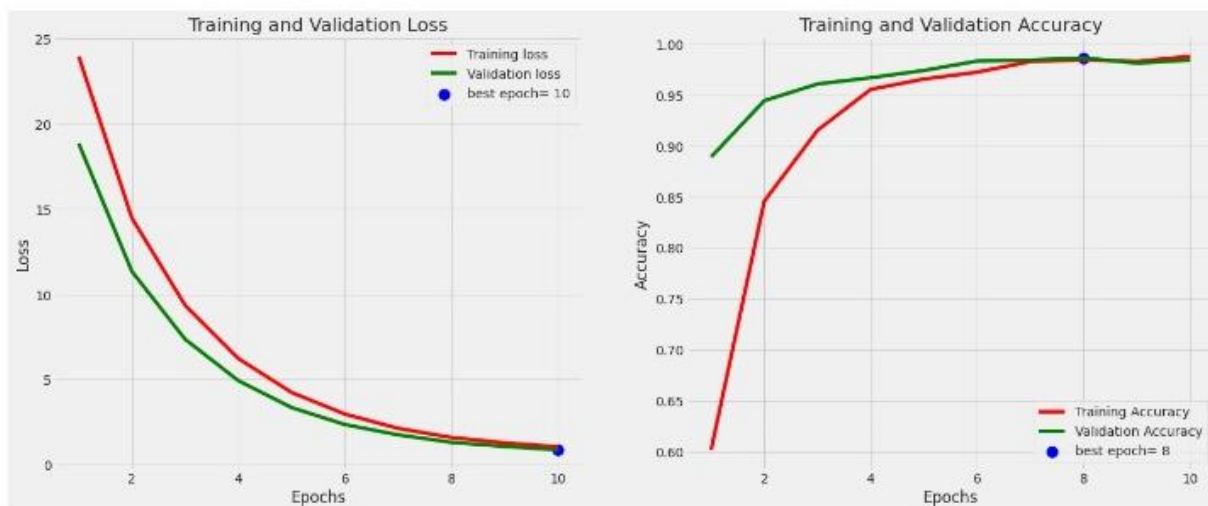
```

Model Loss (Gubici)

```

plt.plot (Epochs,tacc,'r',label= 'Training Accuracy')
plt.plot (Epochs,vacc,'g',label= 'Validation Accuracy')
plt.scatter(index_acc+1 +start_epoch,acc_highest, s=150, c= 'blue',
label=vc_label)
plt.set_title('Training and Validation Accuracy')
plt.set_xlabel('Epochs')
plt.set_ylabel('Accuracy')
plt.legend()
plt.tight_layout
plt.show()

```



Slika 9. Preciznost i gubici neuronske mreže

Vidimo da nam je test preciznost veća od trenažne, do osme epoch-e što znači da nam je model naučio sortirati naše podatke do osme epoch-e, takođe vidimo da su nam gubici u redu. Sve u svemu možemo reći da imamo funkcionalan model do osme epoch-e, ali je potrebno još treniranja da bi tačnost bila veća.

Rezultate možemo još prikazati i u matrici konfuzije

```
plt.figure(figsize=(12, 6))
sns.heatmap(cm, annot=True, vmin=0, fmt="g", cmap="Blues", cbar=False)
plt.xticks(np.arange(class_count) + 0.5, classes, rotation=90)
plt.yticks(np.arange(class_count) + 0.5, classes, rotation=0)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
clr = classification_report(
    y_true, y_pred, target_names=classes, digits=4
```

		Confusion Matrix																	
Actual	Apple_Bad	55	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0
	Apple_Good	0	56	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0
	Apple_mixed	0	0	6	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	Banana_Bad	0	0	0	54	0	1	0	0	0	0	0	0	0	0	0	0	0	0
	Banana_Good	0	0	0	0	55	0	0	0	0	0	0	0	0	0	0	0	0	0
	Banana_mixed	0	0	0	0	1	14	0	0	0	0	0	0	0	0	0	0	0	0
	Guava_Bad	0	0	0	0	0	0	56	0	0	0	0	0	0	0	0	0	0	0
	Guava_Good	0	0	0	0	0	0	0	57	0	0	0	0	0	0	0	0	0	0
	Guava_mixed	0	0	0	0	0	0	0	0	8	0	0	0	0	0	0	0	0	0
	Lemon_mixed	0	0	0	0	0	0	0	0	0	14	0	0	0	0	0	0	0	0
	Lime_Bad	0	0	0	0	0	0	0	0	0	0	54	0	0	0	0	0	0	0
	Lime_Good	0	0	0	0	0	0	0	0	0	0	1	54	0	0	0	0	0	0
	Orange_Bad	0	0	0	0	0	0	0	0	0	0	0	0	58	0	0	0	0	0
	Orange_Good	0	0	0	0	0	0	0	0	0	0	0	0	0	61	0	0	0	0
	Orange_mixed	0	0	0	0	0	0	0	0	0	0	0	0	0	0	6	0	0	0
	Pomegranate_Bad	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	59	0	0
	Pomegranate_Good	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	297	0
	Pomegranate_mixed	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	6
		Apple_Bad	Apple_Good	Apple_mixed	Banana_Bad	Banana_Good	Banana_mixed	Guava_Bad	Guava_Good	Guava_mixed	Lemon_mixed	Lime_Bad	Lime_Good	Orange_Bad	Orange_Good	Orange_mixed	Pomegranate_Bad	Pomegranate_Good	Pomegranate_mixed
		Predicted																	

Slika 10. Matrica konfuzije

Kao što vidimo iz matrice konfuzije najbolji rezultati su se pokazali kod nara dobre kvalitete gdje imamo 297 preklapanja stvarne predviđene slike i preciznost od 0.9966 %. Ukupna tačnost za cjelokupnu klasifikaciju iznosi 0.9939% što je veoma dobro.

Zaključak

Umjetne neuronske mreže su stvarno impresivna metoda u kojim se razvija umjetna inteligencija. Različitim topologijama odnosno dizajnom strukture umjetne neuronske mreže dobivaju se mogućnosti rješavanja različitih problema. Umjetne neuronske mreže se najčešće specijaliziraju za rješavanje određenog problema te nažalost se ne mogu mjeriti s kreativnošću ljudskog uma. Za specijalizirane probleme čak i u nekim segmentima nadilaze ljudske sposobnosti. Umjetne neuronske mreže se svakodnevno koriste prilikom pretraživanja na internetu ili korištenjem modernih tehnologija. Smatram da su umjetne neuronske mreže još u "razvoju" te da će njihovo korištenje biti učestalije u bližnjoj budućnosti. U ovom projektu smo opisali bazu slika s voćem te napravili određene eksperimente koristeći TensorFlow biblioteku kako bi razvrstali slike prema njihovom sadržaju. Ono što je važno za budućnost jeste da se poboljša tačnost neuronske mreže. To uključuje razna podešavanja i izmjene. Druga mogućnost jeste da se zamijene svi slojevi konvolucijskim slojevima. Pokazalo se da to daje određena poboljšanja u odnosu na mreže koje imaju potpuno povezane slojeve u svojoj strukturi. Posljedica toga je povećanje broja parametara za mrežu. Još jedan način jeste da se proširi skup podataka kako bi se uključilo više vrsta voća.

Literatura:

- [1] X. Chen, G. Zhou, A. Chen, L. Pu, W. Chen The fruit classification algorithm based on the multi-optimization convolutional neural network
- [2] S. Behera, A. Rath, P. Sethy Maturity status classification of papaya fruits based on machine learning and transfer learning approach
- [3] H. Ayaz, E. Rodríguez-Esparza, M. Ahmad, D. Oliva, M. Pérez-Cisneros, R. Sarkar Classification of apple disease based on non-linear deep features
- [4] M. Momenya, A. Jahanbakhshib, K. Jafarnejhadc, Yu-Dong Zhang Accurate classification of cherry fruit using deep CNN based on hybrid pooling approach
- [5] <https://www.sciencedirect.com/science/article/pii/S2352340921009616#bib0004>